**Vision of the Department**

*To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.*

**Mission of the Department**

*To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

**Session 2025-2026**

| | |
|---|---|
| **Vision:** *To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration* | **Mission:** *To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.* |

**Program Educational Objectives of the program (PEO):** (broad statements that describe the professional and career accomplishments)

| | | | |
|---|---|---|---|
| PEO1 | **Preparation** | **P: Preparation** | **Pep-CL abbreviation pronounce as Pep-si-lL easy to recall** |
| PEO2 | **Core Competence** | **E: Environment (Learning Environment)** | |
| PEO3 | **Breadth** | **P: Professionalism** | |
| PEO4 | **Professionalism** | **C: Core Competence** | |
| PEO5 | **Learning Environment** | **L: Breadth (Learning in diverse areas)** | |

**Program Outcomes (PO):** (statements that describe what a student should be able to do and know by the end of a program)

**Keywords of POs:**
Engineering knowledge,Problem analysis, Design/development of solutions,Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World,Ethics, Individual and Collaborative Team work,Communication, Project Management and Finance, Life-Long Learning

**PSO Keywords:** Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

**Integrity:** I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

**Sheikh Wasimuddin**
**Name and Signature of Student and Date**
(Signature and Date in Handwritten)

22/1/26

Nagar YuwakShikshanSanstha's
YeshwantraoChavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Technology

**Vision of the Department**

*To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.*

**Mission of the Department**

*To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

| **Session** | **2025-26 (EVEN)** | **Course Name** | **JAVA FSD Lab** |
|---|---|---|---|
| **Semester** | **6** | **Course Code** | **23IOT1607** |
| **Roll No** | **63** | **Name of Student** | **Sheikh Wasimuddin** |

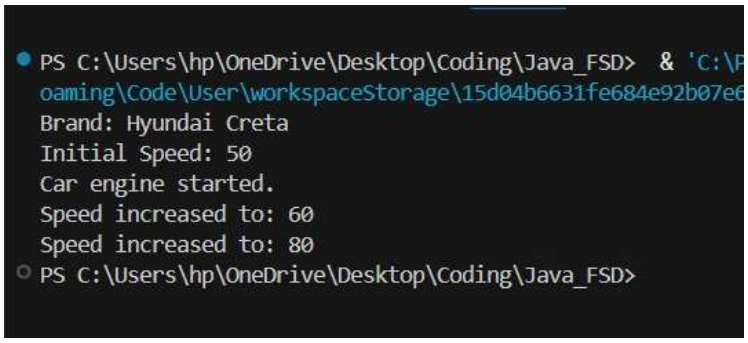
| | |
|---|---|
| Practical Number | **01** |
| Course Outcome | 1. Apply OOP concepts to solve the real life problem.<br>2. Use the database concepts to integrate in giving OOP oriented solution.<br>3. Develop full-stack applications integrating frontend, backend, and database with security.<br>4. Design and develop mini-project with frontend, backend, and database integration, including authentication and testing. |
| Aim | Implement the basic object oriented concepts:<br>1. Encapsulation<br>2. Inheritance<br>3. Abstraction<br>4. Polymorphism |
| Problem Definition | Vehicle Management System to demonstrate the four pillars of Object-Oriented Programming (OOP) in Java. |
| Theory<br>(100 words) | The four pillars of Object-Oriented Programming are clearly demonstrated in this example.<br>**Abstraction** is achieved through the Vehicle class, which represents the general concept of a vehicle and defines common behaviors without revealing the specific implementation details, such as how the engine starts.<br>**Inheritance** is shown when the Car class extends the Vehicle class, allowing it to inherit properties and methods and enabling code reuse.<br>**Encapsulation** is implemented by declaring the brand and speed variables as private, restricting direct access to them and allowing interaction only through public getter and setter methods, which ensures data protection and controlled access.<br>**Polymorphism** is demonstrated in two forms: runtime polymorphism occurs when the Car class overrides the startEngine() method to provide its own implementation, while compile-time polymorphism is achieved through method overloading by defining multiple accelerate() methods with different parameters. |

| | |
|---|---|
| Procedure and Execution<br><br>(100 Words) | Algorithm:<br>Step 1: Start the program.<br>Step 2: Define an Abstract base class (e.g., Vehicle) to represent the general concept<br>and hide implementation details of specific methods.<br>Step 3: Use Encapsulation by declaring data members<br>(like brand or speed)<br>as private and creating public getter and setter methods to control access.<br>Step 4: Implement Inheritance by creating a child class (e.g., Car) that extends the<br>base class to reuse existing code.<br>Step 5: Implement Compile-time Polymorphism by defining overloaded methods<br>(e.g., accelerate()) with the same name but different parameters.<br>Step 6: Implement Runtime Polymorphism by overriding the base class methods in<br>the child class to provide specific behavior.<br><br>Step 7: Create a main class to instantiate the objects and invoke the methods.<br>Step 8: Stop the program |
| | Code:<br>abstract class Vehicle {<br><br>private String brand;<br>private int speed;<br><br>public Vehicle(String brand) {<br>this.brand = brand;<br>this.speed = 0;<br>}<br><br>public String getBrand() {<br>return brand;<br>}<br><br>public void setBrand(String brand) {<br>this.brand = brand;<br>} |

```
    public int getSpeed() {
        return speed;
    }

    public void setSpeed(int speed) {
        this.speed = speed;
    }


    abstract void start();


    void accelerate() {
        speed += 10;
        System.out.println("Speed increased to " + speed);
    }

    void accelerate(int increment) {
        speed += increment;
        System.out.println("Speed increased to " + speed);
    }
}


class Car extends Vehicle {

    public Car(String brand) {
        super(brand);
    }


    @Override
    void start() {
        System.out.println(getBrand() + " car starts with a key.");
    }
}


public class VehicleManagementSystem {
    public static void main(String[] args) {


        Vehicle v = new Car("Hyundai Creta");
```

```
        v.start();
        v.accelerate();
        v.accelerate(20);

        System.out.println("Final Speed: " + v.getSpeed());
    }
}
```

Output:
+




| | |
|---|---|
| Output Analysis | The program successfully demonstrates key Object-Oriented Programming concepts by creating a Car object and starting its engine using runtime polymorphism, which allows the appropriate method implementation to be invoked at execution time. It also displays encapsulated vehicle details such as the brand and initial speed, ensuring controlled access to data. Additionally, the vehicle's speed is increased using overloaded accelerate() methods, effectively illustrating compile-time polymorphism by invoking different method versions based on the parameters passed. |
| Link of student Github profile where lab assignment has been uploaded | https://github.com/sheikhwasimuddin/Java-FSD |
| Conclusion | This program effectively showcases the practical application of Object-Oriented Programming principles in Java. By using encapsulation, polymorphism (both compile-time and runtime), and object creation, it models real-world vehicle behavior in a clear and structured manner. The successful execution and correct output confirm that the design is well-implemented, making the program easy to understand, maintain, and extend for future enhancements. |

**Vision of the Department**

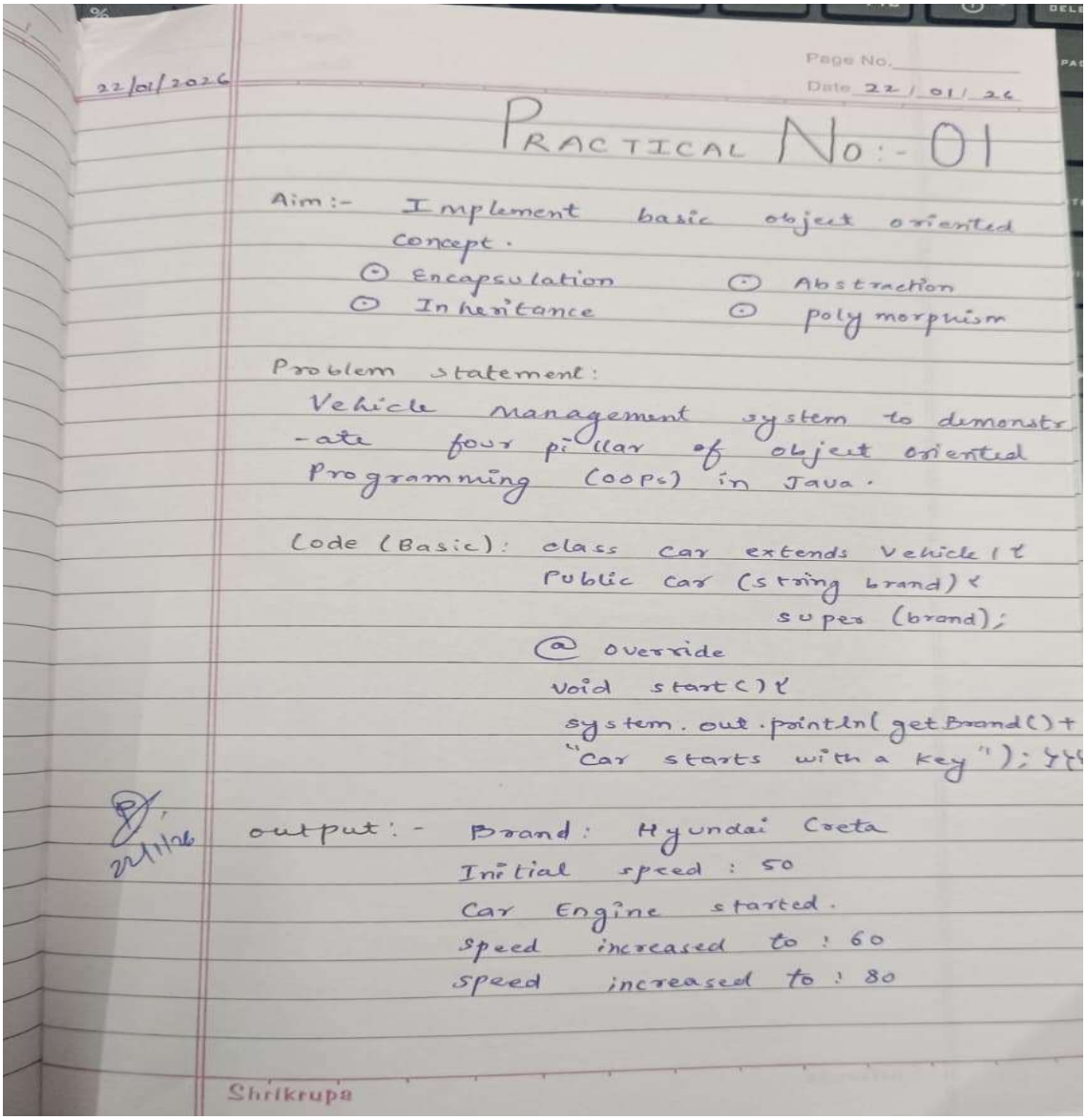
*To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.*

**Mission of the Department**

*To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

| | |
|---|---|
| Plag Report (Similarity index < 12%) | 8% Plagiarism; Exact Match 8%; Partial Match 0%; 92% Unique |
| Date | **22-01-26** |

## Paste here Faculty signed observation page.



22/01/2026

Page No.
Date 22/01/26

PRACTICAL No:- 01

Aim:- Implement basic object oriented concept.
- Encapsulation
- Abstraction
- Inheritance
- polymorphism

Problem statement:
Vehicle management system to demonstrate four pillar of object oriented Programming (OOPs) in Java.

Code (Basic):
```
class Car extends Vehicle {
Public Car (string brand) {
super (brand);
@ Override
Void start() {
system.out.println(getBrand()+
"Car starts with a key"); }}
```

output:- Brand: Hyundai Creta
Initial speed : 50
Car Engine started.
speed increased to : 60
speed increased to : 80

Shrikrupa